

LLMProvider

LLMProvider

Адзін інтэрфейс, 43 пастаўшчыкі — з аўтаматычнымі засцерагальнікамі, паўторамі і маніторынгам стану.

Зводка

LLMProvider — гэта ўніверсальны, паўторна выкарыстоўваемы модуль Go, які вызначае адзіны інтэрфейс LLMProvider разам з прадукцыйнымі шаблонамі адмоваўстойлівасці — засцерагальнік, маніторынг стану, паўтор з затрымкай, лянівая загрузка — і пастаўляе 43 канкрэтныя рэалізацыі пастаўшчыкоў за адным кантрактам, з універсальным адаптарам, сумяшчальным з OpenAI, і сумленным, без жорстка зашытых запасных варыянтаў, механізмам выяўлення мадэляў.

Кароткае апісанне

Паўторна выкарыстоўваемы модуль Go, які прапануе адзін інтэрфейс LLMProvider (Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig) разам з прымітывамі адмоваўстойлівасці — засцерагальнік, манітор стану, паўтор з затрымкай і выпадковым адхіленнем, лянівая ініцыялізацыя — для 43 адаптараў пастаўшчыкоў і ўніверсальнага адаптара, сумяшчальнага з OpenAI. Паточна-бяспечны.

Падрабязнае апісанне

LLMProvider — гэта той абстракцыйны слой, які патрэбен кожнай службе, што спажывае LLM, але які амаль ніхто не стварае належным чынам — непаказная «сантэхніка», што адлучае дэманстрацыю ад сістэмы, якая здольна вытрымаць сутыкненне з рэальным трафікам. Ён вызначае адзіны інтэрфейс, які ўлічвае магчымасці — Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig — каб прыкладны код

меў справу з адным кантрактам незалежна ад таго, які з 43 бэкэндаў адказвае на запыт, а затым дадае аперацыйнае ўмацаванне, якое ператварае кволя звароты да пастаўшчыкоў у нешта, што можна запускар у прадукцыйным асяроддзі без затайвання дыхання. Трохстадыйны засцерагальнік (закрыты → адкрыты → паўадкрыты) празрыста абгортвае любога пастаўшчыка — *уключаючы яго струменевы канал*, дзе пусты струмень правільна лічыцца памылкай — каб адзін нестабільны бэкэнд мог спрацаваць і спыніць падзенне ўсёй службы; цэнтральны `CircuitBreakerManager` адсочвае ўсе засцерагальнікі адначасова. Канфігураваны манітор стану бесперапынна правярае пастаўшчыкоў на станы «здоровы» / «дэградаваны» / «нездоровы» / «невядомы» паводле парогавых значэнняў і інтэрвалаў, каб дэградацыя фіксавалася, а не выяўлялася праз адмову. Логіка паўтораў накладвае экспаненцыяльную затрымку з выпадковым адхіленнем, прымаючы рашэнні на аснове статусу — паўтараць варта толькі тыя памылкі, якія варта паўтараць (429, 5xx, часовыя сеткавыя збоі), і ніколі не марнаваць цыклы на 4xx ці адменены кантэкст — з абмежаваннем затрымак, каб шторм паўтораў не выйшаў з-пад кантролю. А шаблон лянівай ініцыялізацыі адкладае стварэнне кожнага пастаўшчыка да яго першага рэальнага выкарыстання — наўмыснае рашэнне, якое робіць рэгістрацыю ўсіх 43 пастаўшчыкоў фактычна бясплатнай.

Модуль пастаўляе 43 канкрэтныя пакеты пастаўшчыкоў разам з generic адаптарам, сумяшчальным з OpenAI, які рэалізуе поўны інтэрфейс для *любога* канцавога пункта `/v1/chat/completions` — аўтэнтыфікацыя праз Bearer, струменевая перадача праз SSE з карэктнай апрацоўкай `[DONE]` — каб пастаўшчык без прысвечанага пакета заставаўся поўнапраўным удзельнікам з моманту накіравання адаптара на яго URL. Крэдыты вызначаюцца ў адным месцы (`apikeys`, з выкарыстаннем строгай канвенцыі `ApiKey_<Provider>`), што выключае ўвесь клас памылак тыпу «жорстка зашыты ключ прайшоў тэсты, а рэальны ключ так і не падключыўся, і прадукт зламаўся ў прадзе». Выяўленне мадэляў наўмысна, амаль упарта сумленнае: яно запытвае жывыя API пастаўшчыкаў праз кэш з абмежаваннем часу жыцця, і — згодна з кіраваннем — стары ўзровень жорстка зашытых запасных варыянтаў быў цалкам выдалены. Калі жывое выяўленне не спрацоўвае, `LLMProvider` не вяртае *нічога* замест састарэлага каталога, каб кліенту ніколі не трапіў ідэнтыфікатар мадэлі, які выглядае сапраўдным, але не можа быць выкліканы. Усе элементы створаны паточна-бяспечнымі для канкурэнтнага выкарыстання.

Змесціва

Чаму мы яго стварылі

Найўныя званкі LLM правальваюцца на прадукцыйным узроўні — правайдэры абмяжоўваюць хуткасць, дэградаюць ці зусім падаюць, і адзін дрэнны бэкэнд можа пахаваць з сабой усю службу. Каталогі мадэляў змяняюцца, а жорстка запісаныя спісы перадаюць кліентам ідэнтыфікатары, якія ўжо не працуюць. LLMProvider цэнтралізуе інтэрфейс, шаблоны адмоваўстойлівасці і сумленнае выяўленне, каб кожны спажывец атрымліваў устойлівасць да збояў і дакладнасць бясплатна.

Чаму гэта пераварочвае гульню

Гэта зводзіць “інтэграцыю правайдэра LLM” да аднаго кроку — рэалізуйце адзін інтэрфейс ці проста накіруйце ўніверсальны адаптар на канцавы пункт — і бібліятэка аўтаматычна і празрыста абгортвае гэтага правайдэра ў разрыў ланцуга, маніторынг здароўя і паўторныя спробы з экспаненцыяльным затрымкай і джытэрам. Адмоваўстойлівасць перастае быць нечым, што кожная каманда вынаходзіць нанова (дрэнна, пад дэдлайнам, пасля першага збою), і становіцца стандартным паводзінамі бібліятэкі для ўсіх 43 бэкэндаў. Інжынерынг надзейнасці напісаны адзін раз, старанна пратэставаны і перадаецца бясплатна ўсім, хто яго імпартуе.

Што новага

- **Адзін інтэрфейс з улікам магчымасцей** — завяршэнне, стрымінг, здароўе, магчымасці і валідацыя канфігурацыі аб’яднаны ў адзіны кантракт, які аднолькава выконваюць усе бэкэнды.
- **Празрыстае абгортванне разрывальнікам ланцуга — уключаючы стрымінг.** Разрывальнік абараняе канал CompleteStream, а не толькі запыт/адказ, і разглядае пусты стрым як сапраўдны збой — з бяспечнымі ад блакіровак апавяшчэннямі слухачоў без блакіроўкі.
- **43 пакеты правайдэраў + універсальны адаптар, сумяшчальны з OpenAI** — спецыялізаваныя пакеты застаюцца лёгкімі, і любы неўключаны вендар, які падтрымлівае /v1/chat/completions, пачне працаваць, як толькі вы накіруеце на яго адаптар.
- **Адзіная крыніца даверу для дадзеных аўтэнтыфікацыі (apikeys)** — толькі адно месца чытае пераменныя асяроддзя ApiKey_<Provider>, што структурна выключае нязгоду “зьялёныя тэсты, паломаны прадукт” замест простага папярэджання пра яе.

- **Сумленнае выяўленне мадэляў (без жорсткіх запасных варыянтаў)** — жывыя API правайдэраў за кешам з TTL; пры збоі вяртае nil, а не састарэлы ці сфальсіфікаваны каталог, які выдае невыклікальныя ідэнтыфікатары.
- **Лянівая ініцыялізацыя з sync.Once** — стварэнне адкладаецца да першага выкарыстання, таму рэгістрацыя ўсіх 43 правайдэраў амаль нічога не каштуе, пакуль вы сапраўды не звярнецеся да аднаго з іх.
- **Антыблефны стэк Challenge з падтрымкай некалькіх лакаляў** — рэальны ранэр, які правярае паводзіны разрывальніка, маніторынгу здароўя і паўторных спроб у пяці лакалях, з абмежаваннем праз мутацыйнае тэставанне (немутаваны код павінен завяршацца з кодам 0; уведзеная мутацыя павінна прыводзіць да выхаду з кодам 99), каб прайшоўшы набор тэстаў гарантаваў працаздольнасць.

Найбольшыя тэхнічныя выклікі і як мы іх пераадолелі

- **Каскадныя збоі правайдэраў.** Адзін нястабільны бэкэнд не павінен цягнуць за сабой усю службу. Вырашана з дапамогай трохстаднага разрывальніка ланцуга (закрыты → адкрыты → паўадкрыты), які празрыста абгортвае любога правайдэра *і яго стрым*, пераходзіць у адкрыты стан пры сталым збоі, правярае аднаўленне ў паўадкрытым стане і цэнтралізавана каардынуецца праз CircuitBreakerManager.
- **Часовыя памылкі і абмежаванні хуткасці.** Вырашана з дапамогай экспаненцыяльнай затрымкі з джытэрам, якая ўлічвае статус — $\min(\text{InitialDelay} \cdot \text{Multiplier}^{(n-1)}, \text{MaxDelay}) \pm \text{jitter}$ — каб паўторныя спробы распаўсюджваліся, а не сінхранізаваліся ў “статак”. Паўтараюцца толькі тыя памылкі, якія варта паўтараць (429, 500, 502, 503, 504 і сеткавыя памылкі), і адхіляюцца спробы на скасаваны кантэкст ці любы іншы 4xx.
- **Маштабаванне да многіх зарэгістраваных, але невыкарыстаных правайдэраў.** З 43 зарэгістраванымі правайдэрамі, але толькі некалькімі актыўнымі ў кожнай службе, імклівая ініцыялізацыя была б чыстым марнаваннем. Вырашана з дапамогай лянівай ініцыялізацыі пад аховай sync.Once, каб толькі тыя правайдэры, да якіх вы сапраўды звяртаецеся, плацілі за сваё падрыхтоўку.
- **Выдача несапраўдных ідэнтыфікатараў мадэляў.** Вырашана поўным выдаленнем узроўню жорстка запісаных запасных варыянтаў выяўлення (згодна з CONST-036) і вяртаннем нічога пры збоі жывога выяўлення — плюс абароненае капіраванне пры вяртанні, каб кліент не мог змяніць кеш ці канкураваць з іншым чытачом. Сумленнасць забяспечваецца структурна, а не паводле пагаднення.

- **Стрымінг і карэктнасць канкурэнтнасці.** Схаваная памылка — гэта дэдлок паміж блакіроўкай разрываўніка і яго функцыямі-слухачамі. Вырашана з дапамогай здымкаў слухачоў і апавяшчэнняў без блакіроўкі з тайм-аўтам у 5 секунд, а таксама разблакоўкай перад апавяшчэннем аб скіданні — прычым усе кампаненты створаны для канкурэнтнага выкарыстання і правераны наборам - race.

Змесціва

Тэхналагічны стэк

- **Go (1.25.3)** — абраны за першакласную канкурэнтнасць, статычныя бінарныя файлы і магутную стандартную бібліятэку; змяшчае модуль, інтэрфейс, усе прымітывы адмоваўстойлівасці і 43 адаптары.
- **net/http (stdlib)** — наўмысна свабодны ад залежнасцей HTTP: забяспечвае працу кліентаў для кожнага правайдэра, універсальнага адаптара, сумяшчальнага з OpenAI, і жывых запытаў на выяўленне, таму няма патрэбы аўдытаваць ці абнаўляць транспарт трэціх асоб.
- **logrus** — структурызаванае лагіраванне з кантролем узроўняў менавіта там, дзе аператарам патрэбна бачнасць: у пераходах станаў брэйкера і на шляху выяўлення.
- **testify** — кіруе тэставай камплектацыяй і, што крытычна, фіксацыяй мутацыйных галін, дзякуючы якой паспяховае выкананне мае рэальны сэнс.
- **yaml.v3** — апрацоўвае бандлы інтэрнацыяналізацыі і канфігурацыі ў фармаце, які застаецца зручным для ручной рэдакцыі.
- **digital.vasic.models** — агульныя тыпы LLMRequest / LLMResponse / ProviderCapabilities, якія захоўваюцца ў адным месцы, каб усе адаптары размаўлялі на адной мове (дакументаваная залежнасць часу выканання).
- **Уласныя пакеты** — circuit, health, retry, apikeys, discovery, providers/ (43 вендары + generic) і il8n: паверхня адмоваўстойлівасці і інтэграцыі падзелена на невялікія, незалежна тэставаныя блокі замест аднаго маналіту.
- **.env + ~/api_keys.sh (канвенцыя ApiKey_<Provider>)** — адзіная, недвухсэнсоўная крыніца сапраўднасці дадзеных для аўтэнтыфікацыі, каб ключы падключаліся аднолькава ў тэстах і на прадукцыі.
- **Makefile race suite (-race -p 1) + Challenge runner** — аснова, якая не дазваляе блефаваць: дэтэктар гонкі пацвярджае карэктнасць канкурэнтнасці, а Challenge runner правярае рэальную працу ў рэжымах хаосу, DDoS-атак, маштабавання, стрэсу, жывога выяўлення і сцэнараў без прыпынку.

Стан і заўвагі аб шчырасці

- **Стан: бэта.** Дэкапліраваны модуль шматразовага выкарыстання; рэпазіторый GitHub з’яўляецца публічным.
- **Ліцэнзія: пакуль не вызначана.** Супярэчнасці — doc.go паказвае на MIT, у той час як існуе файл LICENSE ў стылі Apache-2.0 — пераканайцеся перад публікацыяй.
- LLMsVerifier — гэта асноўная крыніца праўдзівай інфармацыі аб кананічным каталогу мадэляў. Маніфест helix-deps.yaml выглядае састарэлым (заяўляе deps: [], хоць дакументацыя паказвае залежнасць ад digital.vasic.models); “Tier 2 (models.dev)” у сістэме выяўлення — гэта запланаваны, але яшчэ не актыўны заглушавы варыянт.

Прыярытэтны ўзровень: Helix-primary (кластар LLM-infrastructure — дэкапліраваны модуль шматразовага выкарыстання). Ранжыруецца пасля HelixTrack.